



Recovering Markov models from closed-loop data[☆]

Jonathan P. Epperlein^{b,*}, Sergiy Zhuk^b, Robert Shorten^a

^a University College Dublin, Dublin, Ireland

^b IBM Research, Dublin, Ireland



ARTICLE INFO

Article history:

Received 16 November 2017

Received in revised form 3 October 2018

Accepted 11 December 2018

Available online 14 February 2019

ABSTRACT

Situations in which recommender systems are used to augment decision making are becoming prevalent in many application domains. Almost always, these prediction tools (recommenders) are created with a view to affecting behavioural change. Clearly, successful applications actuating behavioural change, affect the original model underpinning the predictor, leading to an inconsistency. This feedback loop is often not considered in standard machine learning techniques which rely upon machine learning/statistical learning machinery. The objective of this paper is to develop tools that recover unbiased user models in the presence of recommenders. More specifically, we assume that we observe a time series which is a trajectory of a Markov chain $\mathbf{R}$ modulated by another Markov chain $\mathbf{S}$, i.e. the transition matrix of $\mathbf{R}$ is unknown and depends on the current state of $\mathbf{S}$. The transition matrix of the latter is also unknown. In other words, at each time instant, $\mathbf{S}$ selects a transition matrix for $\mathbf{R}$ within a given set which consists of known and unknown matrices. The state of $\mathbf{S}$, in turn, depends on the current state of $\mathbf{R}$ thus introducing a feedback loop. We propose an Expectation–Maximisation (EM) type algorithm, which estimates the transition matrices of $\mathbf{S}$ and $\mathbf{R}$. Experimental results are given to demonstrate the efficacy of the approach.

© 2019 Elsevier Ltd. All rights reserved.

1. Introduction

Our starting point for this paper is a frequently encountered problem that arises in the Smart Cities domain. Many decision support/recommender systems that are designed to solve Smart City problems are data-driven: that is data, sometimes in real time, is used to build models to drive the design of recommender systems. Almost always, these data sets are treated as if they were obtained in an open-loop setting, i.e. without recommender influence. However, this is rarely the case and frequently the effects of recommenders are inherent in data sets used for model building (Bottou et al., 2013; Crisostomi, Shorten, & Wirth, 2016; Lazer, Kennedy, King, & Vespignani, 2014; Sinha, Gleich, & Ramani, 2016). This creates new challenges for the design of decision support/recommender systems under feedback. In particular, as engineers, we must take into account the fact that when we make a prediction, then this prediction affects the behaviour of operators (Cosley, Lam, Albert, Konstan, & Riedl, 2003) and this, in turn, changes the data set upon which the original model was built. Clearly, the aforementioned effect is related to classical closed-loop identification, which is itself a mature topic in both control

and economics (Forsell & Ljung, 1999; Stock & Trebbi, 2003; Van Den Hof & Schrama, 1995).

Notwithstanding this fact, and even though avoiding closed-loop effects in the design of recommender systems has been the subject of study in several Smart City applications (Schlote, Chen, & Shorten, 2015; Schlote, King, Crisostomi, & Shorten, 2014), the development of algorithms to identify models in closed loop remains a challenging problem in the context of Smart Cities. This is due to the fact that closed-loop questions that arise in Smart Cities are, for the most part, qualitatively different to those arising in other areas. For example, in control theory closed-loop questions typically arise in the context of deterministic and parametric models subject to noise, whereas in Smart Cities, typical problems are characterised by large-scale data sets that are generated by largely unknown stochastic processes.

Our objective is to consider one such problem class that arises in Smart City related research, where we seek to identify a user model, based on observations obtained when the user is acting under the influence of a recommender. A particular instance of such a problem arises in the automotive domain where drivers are characterised using Markovian models (Epperlein et al., 2018; Krumm, 2008), but where observations are obtained under the influence of recommenders acting on the driver. Motivated by such applications, we seek to develop methods to account for the effect of these recommender systems in data sets. More formally, we shall consider systems with the following structure: the process, which generates the data; the model, which represents

[☆] The material in this paper was not presented at any conference. This paper was recommended for publication in revised form by Associate Editor Oswaldo Luiz V. Costa under the direction of Editor Richard Middleton.

* Corresponding author.

E-mail addresses: jpepperlein@ie.ibm.com (J.P. Epperlein), sergiy.zhuk@ie.ibm.com (S. Zhuk), robert.shorten@ucd.ie (R. Shorten).

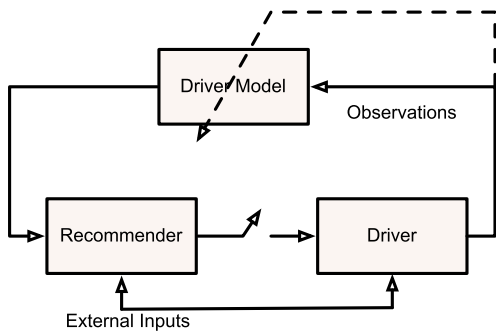


Fig. 1. Automotive recommender architecture.

the behaviour of the process; and the *decision support tool*, which intermittently influences the process. In our setup, data from the process is used to build the model. Typically, the model is used to construct a decision support tool which itself then influences the process directly. This creates a feedback loop in which the process, decision support tool and the model are interconnected in a complicated manner. As a result, the effect of the decision support tool is to bias the data being generated by the process, and consequently to bias any model that is constructed naively from the data.

To provide a little more context, and to return to the automotive example, we now illustrate such effects by means of the following application that we have developed in the context of our automotive research.¹ Consider a driver who drives a car regularly. In order to design a recommender system for this driver we would like to build a model of his/her behaviour. For example, in order to warn the driver of, say, roadworks, along a likely route, we might use this model to predict the route of the driver. A schematic of the proposed in-car architecture is depicted in Fig. 1. The recommender uses a model of driver behaviour to issue intermittent recommendations. Observations of driver behaviour are then used to build a refined driver model which in turn is used as an input to the recommender system. Clearly, the effect of the recommender is to bias the driver model over time, thus eventually rendering the latter ineffective as an input to the recommender. The problems are exacerbated in many practical systems due to the presence of several unknown third-party recommender systems (Google Maps, Siri etc.), and by the fact that the *driver model* may operate from *birth-to-death*² in closed loop. This latter fact makes it difficult, or impossible, to even estimate an initial model of driver behaviour. Clearly, in such applications it is absolutely necessary to develop techniques that extract the behaviour of the driver while under the influence of the feedback from a number of recommender systems. The results presented in this paper represent our first small step in this direction.

1.1. General comments on related research directions

Dealing with bias arising from closed-loop behaviour is a problem that has arisen in several application domains. In fact, in control theory, the related topic of closed-loop identification is considered to be a very mature area (Norton, 2009; Söderström & Stoica, 1989). Roughly speaking, this topic is concerned with building models of dynamic systems while they are being regulated by a controller. A related scenario arises in some adaptive

systems when the controller itself is being adjusted on the basis of the dynamical systems model. As in our example, the controller action will bias the estimation of the model parameters. While many established techniques in control theory exist for dealing with such effects, these typically exploit known properties of the process noise and an assumed model structure to un-bias the estimates. Typically, structures such as ARMAX models are assumed to capture the nature of the system dynamics. Recent work on *intermittent feedback* (Loram, Gollee, Lakie, & Gawthrop, 2011) is also related in spirit to these approaches where control design techniques to deal with feedback loops that are intermittently broken are developed. Before proceeding it is worth noting that closed-loop effects have also been explored in the economics literature (Angrist & Pischke, 2008; Pearl, 2000; Stock & Trebbi, 2003; Varian, 2016).

More recently, several authors in the context of Smart Cities and recommender systems (Crisostomi et al., 2016; Sinha et al., 2016), have realised that closed-loop effects represent a fundamental challenge in the design of recommender systems. In Crisostomi et al. (2016) the authors discuss the inherent *closed-loop* nature of data-sets in cities, and in Sinha et al. (2016) explicitly discuss the influence of feedback on the fidelity of recommender systems. As an example of a specific result, Sinha et al. (2016) present an empirical technique for collaborative filtering to recover user rankings in the presence of a recommender under an assumed interaction model between user and recommender, which is similar in spirit to the aforementioned problem of constructing models from data possibly biased by a recommender, but does not consider sequential models.

The work reported here is closely aligned with stochastic models, unlike most of the approaches outlined in the first paragraph of Section 1.1. Specifically, in this paper we are interested in reconstructing Markov models that are operating under the influence of a recommender. To this end we assume that (i) *recommenders and users can both be modelled by Markov chains*, and (ii) *recommendations are either accepted fully or have no influence at all, i.e. every decision is made by either the user or the recommender, never by a combination of the two*. We note here that the Markovian assumption of user and recommender behaviour is convenient for many applications: for example, in the automotive domains (Krumm, 2008; Lassoued et al., 2017).

In this context, the present work is also related to classes of *mixture* and *latent variable models*, such as well-known hidden Markov (HMM) and mixture-of-experts (ME) models (Jacobs, Jordan, Nowlan, & Hinton, 1991; Meila & Jordan, 1996). In the latter case, a latent Markov chain selects from a set of parametrisations of a visible process, however there is no closed-loop modulation (i.e. the modulated visible process is not allowed to in turn modulate the modulating latent process), and the visible process is static, subject to noise, not a Markov chain itself. Our work is most related to Ephraim and Roberts (2009) but again with the important distinction of closed-loop modulation. A similar concept of regime switching time series models is used in econometrics (Lange & Rahbek, 2009): these models allow parameters of the conditional mean and variance to vary according to some finite-valued stochastic process with states or regimes. However, the observations are assumed to be generated by a deterministic process with random noise, and the latent (switching) process is either a Markov chain independent of the past observations or is a deterministic function of the past observations. In contrast, we introduce the closed-loop modulation as discussed above. Yet another related model is Markov jump linear systems, see e.g. Costa, Fragoso, and Marques (2006), where a (latent, autonomous) Markov chain selects the parameters of a (visible) dynamical system, whereas in our case, the visible part is a stochastic process on a discrete state space and it can modulate the latent process.

¹ <https://www.youtube.com/watch?v=KUKxZZByIUM>.

² By that we mean that the driver *always* operates under the potential influence of a recommender. Thus, given any observation, we do not know whether the recommender is acting, or the driver.

More specific technical comments to place our work in the context of reconstructing Markov models from data, and a brief discussion of practical issues including *identifiability* and convergence speed in terms of the number of samples are given below in Section 2.3 after the formal description of the proposed closed-loop Markov-modulated Markov chain models.

1.2. Preliminaries

Notation. To compactly represent discrete state spaces we write $[N] := \{1, \dots, N\}$. For a function $M : [N] \rightarrow \mathbb{R}^{m \times n}$ mapping such a discrete finite set to a set of matrices, we refer to each value $M(k) \in \mathbb{R}^{m \times n}$ as a *page* of M . Matrices will be denoted by capital letters, their elements by the same letter in lower case, and we denote the set of $n \times m$ row-stochastic matrices, i.e. matrices with non-negative entries such that every row sums up to 1, by $\mathcal{M}^{n \times m}$, and $\mathcal{M}^m := \mathcal{M}^{m \times m}$. For compatible matrices, $M \otimes N$ is the Kronecker product, and $M \circ N$ denotes the Hadamard (or element-wise) product. A *partition* Γ of $[N]$ is a set $\{\Gamma_1, \dots, \Gamma_p\}$ such that $\Gamma_i \subseteq [N]$, $\cup_i \Gamma_i = [N]$, $\Gamma_i \cap \Gamma_j = \{\}$ $\forall i \neq j$. Each partition then also defines a membership function $\gamma : [N] \rightarrow [p]$ by $\gamma(i) := k$ such that $i \in \Gamma_k$. We write $P(W = w)$ for the probability of the event that a realisation of the discrete random variable W equals w , and $P(W = w \mid V = v)$ the probability of that same event $W = w$ conditioned on the event $V = v$. We shall denote random variables by capital letters and, where appropriate, their realisations by the same letter in lower case. For convenience we will sometimes write $P(w \mid v)$ instead of $P(W = w \mid V = v)$ if there is no risk of ambiguity, and, for a set of parameters μ parametrisng a probability distribution, $P(W = w \mid \mu)$ is taken to denote the probability of the event $W = w$ if the parameters are set to μ .

Markov chains herein are sequences of random variables $\{X_t\}_t$ indexed by the time $t \in \mathbb{N} = \{0, 1, 2, \dots\}$. The realisation $x_t \in [N]$ of X_t is the *state* of the Markov chain at time t , and $[N]$ is its *state space*. The probability distribution of X_0 is denoted by π_0 , and the probability distribution of each following state is given by $P(X_t = j \mid X_{t-1} = i) = a_{ij}$; the matrix $A \in \mathcal{M}^N$ with entries a_{ij} is the *transition probability matrix*.

2. Problem statement and model

As noted above, we assume that the driver and the recommender are Markovian. Given a possibly incomplete description of Markov chains modelling the recommender systems, and no knowledge of when these systems are engaged, our aim is to *estimate the probability transition matrix of the Markov chain representing the driver*, and the levels of engagement of each recommender, using only observed data. In what follows we formalise this setup, and give an expectation–maximisation (EM) algorithm to estimate the parameters of the unknown driver model.

2.1. “Open-loop” Markov-modulated Markov chains

Consider a Markov chain $\mathbf{R}$ with state space $[R]$ and state $r_t \in [R]$, in which the transition probabilities

$$P(R_t = j \mid R_{t-1} = i, S_{t-1} = s) = a_{ij}^R(s) \quad (1)$$

depend on a *latent* random variable S_t . We can say that the Markov chain is modulated by the random variable S_t , and if S_t is itself the state of another Markov chain $\mathbf{S}$ with transition matrix A^S and state space $[S]$, then we are dealing with a *Markov-modulated Markov chain*; Markov modulation is an established model in the literature on inhomogeneous stochastic processes, see e.g. Ephraim and Roberts (2009).

Algorithm 1 Trajectory generation for a given cl3MC.

```

1: procedure CL3MC TRAJECTORY( $\mu = (\pi^R, \pi^S, A^R, A^S; \Gamma), T$ )
   Initialization
2:   draw  $s_0$  from  $\pi^S$ ,  $r_0$  from  $\pi^R$ 
3:    $t \leftarrow 0$ 
   Iteration
4:   while  $t \leq T - 1$  do
     Transition in S
5:      $\gamma' \leftarrow \gamma(r_t)$  ▷ active page in S
6:     draw  $s_{t+1}$  from  $[a_{s_t 1}^S(\gamma') \dots a_{s_t s}^S(\gamma')]$ 
     Transition in R
7:     draw  $r_{t+1}$  from  $[a_{r_t 1}^R(s_t) \dots a_{r_t R}^R(s_t)]$ 
8:   return  $\mathcal{O} = (r_0 \dots r_T)$ 

```

Formally, the Markov-modulated Markov chain is defined by the tuple $\mu = (\pi^R, \pi^S, A^R(\cdot), A^S)$, where $A^R : [S] \rightarrow \mathcal{M}^R$ and π^S, π^R denote the distributions of S_0 and R_0 , respectively. That means for instance that if $\mathbf{S}$ only has a single state s , μ is a regular Markov chain with transition matrix $A^R(s)$ and initial probability π^R . We assume that we observe the state of $\mathbf{R}$, but not the state of $\mathbf{S}$.

Because the transition probabilities in the latent Markov chain $\mathbf{S}$ do not depend on the state of the visible chain $\mathbf{R}$, we refer to μ as an *open-loop* Markov-modulated Markov chain (ol3MC) to distinguish it from what follows. This models the case when the switching between the transition matrices A^R occurs independently of the current state r_t of $\mathbf{R}$.

The joint process $Q_t = (S_t, R_t)$ has transition probabilities

$$\begin{aligned}
 P(Q_t = (s', r') \mid Q_{t-1} = (s, r)) \\
 &= P(R_t = r' \mid R_{t-1} = r, S_{t-1} = s, S_t \neq s') \\
 &\times P(S_t = s' \mid R_{t-1} = r, S_{t-1} = s) = a_{rr'}^R(s) a_{ss'}^S,
 \end{aligned}$$

where the first cancellation means that the decision at time $t - 1$ is not influenced by the state of the modulating random variable at time t , and the second cancellation follows from the open-loop assumption, i.e. that the modulating Markov chain $\mathbf{S}$ evolves independently of $\mathbf{R}$. The estimation of $\mathbf{S}$ and $\mathbf{R}$ for the case of continuous time ol3MCs has been discussed in Ephraim and Roberts (2009).

Remark. We are dealing with the case when the data consists of a finite time series of observations of a single trajectory $(r_0 r_1 \dots r_T)$ of the Markov chain $\mathbf{R}$ and no (estimate of the) distribution of R_t is available. While if the distributions are available, standard methods of state–space identification apply, here the estimation of the parameters of μ requires statistical methods such as maximum likelihood estimation.

2.2. Closed-loop Markov-modulated Markov chains

As a generalisation, we consider the case where $\mathbf{S}$ is dependent on the state of $\mathbf{R}$: that is, the probabilities of transitioning from one state s to another state s' then do depend on what the current state r_t is. We will be referring to this as a *closed-loop Markov-modulated Markov chain* or *cl3MC* for short. A cl3MC can be used to model that one transition matrix $A^R(s)$ might be more likely to be switched to in some regions of the visible state space $[R]$, or that switching can only occur when the system is in specific configurations. This is exactly the situation which arises in our automotive example, see Section 4.2.

Formally, we now also allow for the latent Markov chain $\mathbf{S}$ to be modulated by the current state of the visible chain $\mathbf{R}$. To keep the developments general, assume that – instead of one page of A^S corresponding to *each* state of $\mathbf{R}$ – there is a partition $\Gamma =$

$\{\Gamma_1, \dots, \Gamma_p\}$ of $[R]$ such that there is a page in A^S for each Γ_i . Hence, we now have $A^S : [p] \rightarrow \mathcal{M}^S$, with

$$P(S_t = j \mid S_{t-1} = i, R_{t-1} = r) = a_{ij}^S(\gamma(r)).$$

The open-loop case then corresponds to $\Gamma = \{[R]\}$ (i.e. $p = 1$ and $\gamma(r) \equiv 1$) and the joint process $Q_t = (S_t, R_t)$ has transition probabilities (compare to the open-loop formula above):

$$\begin{aligned} P(Q_t = (s', r') \mid Q_{t-1} = (s, r)) \\ &= P(R_t = r' \mid R_{t-1} = r, S_{t-1} = s) \\ &\times P(S_t = s' \mid R_{t-1} = r, S_{t-1} = s) \\ &= a_{rr'}^R(s) a_{ss'}^S(\gamma(r)). \end{aligned} \quad (2)$$

Such a cl3MC is represented by a tuple $\mu = (\pi^R, \pi^S, A^R(\cdot), A^S(\cdot); \Gamma)$, where now, $A^S(\cdot)$ has pages, too.

To further illustrate the operation of a cl3MC model, Algorithm 1 details how a realisation of the stochastic process described by it, i.e. a trajectory, is generated.

2.3. Relationship with Hidden Markov Models

There is a close relationship between closed-loop Markov modulated Markov chains and Hidden Markov Models (HMMs). Formally:

Proposition 1. $\mu = (\pi^R, \pi^S, A^R(\cdot), A^S(\cdot); \Gamma)$ defines the same visible process $\{R_t\}$ as the Hidden Markov Model $\lambda = (\pi, W, B)$, with

$$\pi = \pi^S \otimes \pi^R, \quad B = 1_S \otimes I_R,$$

$$w_{ij} = a_{[i/R][j/R]}^S(\gamma(j_R(i))) a_{j_R(i)j_R(j)}^R([i/R]),$$

where $i, j = 1, \dots, RS$, hence $W \in \mathcal{M}^{RS, RS}$.

Here, $j_R(k) := (k-1) \bmod R + 1$ and $[p] := \inf\{k \in \mathbb{Z} \mid k \geq p\}$. The proof of Proposition 1 is mainly algebraic and can be found in Epperlein, Shorten, and Zhuk (2017).

Remark. While Proposition 1 maps a given cl3MC to an HMM which from the outside looks the same as, this mapping is not reversible: not every HMM represents a cl3MC, and most importantly, parameter estimation algorithms such as the standard Baum–Welch algorithm cannot be used to estimate the parameters of a cl3MC, because they do not “respect the structure” of the matrix W : the HMM λ is defined by $(RS)^2 - RS + (RS - 1)$ free parameters (the entries of W and the entries of π with the stochasticity constraints taken into account), whereas the corresponding cl3MC requires only $(pS - 1)(S - 1) + (SR - 1)(R - 1)$ parameters.³ Hence, it is not possible to estimate the parameters of λ and then compute the ones of μ ; instead, we develop an EM-algorithm to estimate the parameters of μ directly in Section 3.3.

Identifiability: Given the close relationship between cl3MCs and HMMs outlined above, one should expect that identifiability issues for cl3MCs bear close resemblance to those of HMMs. By identifiability we mean the following: assume that $(r_1 \dots r_T)$ has been generated by the “true model” μ^{true} ; under which conditions and in what sense will the estimate μ^{est} converge to μ^{true} if $T \rightarrow \infty$? For HMMs this question was partially answered in Petrie (1969), namely it was shown that there is an open, full-measure subset U of all HMMs, such that the sequence of estimates of the BW algorithm converges to λ^{true} (or a trivial permutation of it), provided the starting model is chosen within U , $\lambda_i^{\text{true}} > \delta > 0$ and $T \rightarrow \infty$. However, the structure of U and convergence speed in terms of

the number of samples were not described, and, to the best of our knowledge, these questions are still open.

For cl3MCs, similarly and trivially, any permutation of A^S and the corresponding pages of A^R , which amounts to relabelling the hidden states s , yields the same visible process. However, there are examples of sets of HMMs λ , which are not permutations of each other, yet generate the same observable process; see Blackwell and Koopmans (1957) and Gilbert (1959). Interestingly, those examples involve the special case of *partially observable* Markov chains, a subclass of HMMs with emissions matrices B having entries that are either 1 or 0. Comparing to Proposition 1, a cl3MC has close correspondence to an HMM of this class. This suggests that, in practice, the set of maximisers of the likelihood may be wider than the aforementioned set of permutations of μ^{true} ; our numerical experiments in Section 4 also suggest that, in general, we cannot recover the true model μ^{true} , even up to trivial permutations, from observing only trajectories of $\mathbf{R}$. However, in the case of partial knowledge of elements of A^R we can recover A^S and the unknown portion of A^R . Hence, for the “driver-recommender” problem the proposed method is of practical value. Estimates of the minimum amount of prior knowledge necessary are the subject of future research.

3. Likelihood and parameter estimation

In this section we develop an iterative algorithm to estimate the parameters of a cl3MC $\mu = (\pi^R, \pi^S, A^S(\cdot), A^R(\cdot); \Gamma)$ given a sequence of observations $(r_0 r_1 r_2 \dots r_T)$, a partition $\Gamma = \{\Gamma_1, \dots, \Gamma_p\}$ of $[R]$ and the size S of the state space of $\mathbf{S}$. The derivation is close in spirit to the classical Baum–Welch (BW) algorithm (see e.g. Rabiner, 1989 and the numerous references therein): our algorithm maximises at every iteration a lower bound on the likelihood improvement, and gives rise to re-estimation formulae (14) that utilise forward and backward variables which differ in subtle ways from the ones of the BW algorithm.

3.1. Likelihood of μ , forward- and backward variables

Since the estimate to be obtained is a maximum likelihood (ML) estimate, the efficient computation of the likelihood of a given cl3MC μ plays a central role in what follows. For a given μ , the joint probability of sequences $(r_0 r_1 \dots r_T)$ and $(s_0 s_1 \dots s_T)$ being the trajectories of the visible Markov chain $\mathbf{R}$ and latent Markov chain $\mathbf{S}$ is

$$\begin{aligned} P(s_0, \dots, s_T, r_0, \dots, r_T \mid \mu) \\ &= \pi_{r_0}^R \pi_{s_0}^S \prod_{t=1}^T P(s_t, r_t \mid s_{t-1}, r_{t-1}, \mu) \\ &= \pi_{r_0}^R \pi_{s_0}^S \prod_{t=1}^T a_{s_{t-1}s_t}^S(\gamma(r_{t-1})) a_{r_{t-1}r_t}^R(s_{t-1}) \end{aligned} \quad (3)$$

where the last equality follows by (2). This allows us to compute the probability of observing a sequence $(r_0 r_1 \dots r_T)$ given μ as follows:

$$\begin{aligned} P(r_0 r_1 \dots r_T \mid \mu) \\ &= \sum_{s_0 \in [S] \dots s_T \in [S]} P(s_0, \dots, s_T, r_0, \dots, r_T \mid \mu) \\ &= \pi_{r_0}^R \sum_{s_0 \dots s_T} \pi_{s_0}^S \prod_{t=1}^T a_{s_{t-1}s_t}^S(\gamma(r_{t-1})) a_{r_{t-1}r_t}^R(s_{t-1}) \\ &=: \ell(\mu) \end{aligned} \quad (4)$$

³ Note that $(pS - 1)(S - 1) + (SR - 1)(R - 1) < (RS)^2 - 1$ for $R + S > 2$.

where $\mu \mapsto \ell(\mu)$ is the likelihood of the model μ . Computation using this direct expression requires on the order of $2 \times T \times S^T$ operations, and is hence not feasible for large T . Instead, we define the *forward variable* α_t with S elements

$$\alpha_t(i) := P(S_t = i, R_0 = r_0, \dots, R_t = r_t \mid \mu) \quad (5)$$

which can be computed iteratively as follows: $\alpha_0(j) = \pi_j^S \pi_{r_0}^R$ and $\alpha_t(j) = \sum_{i=1}^S \alpha_{t-1}(i) a_{ij}^S(\gamma(r_{t-1})) a_{r_{t-1}r_t}^R(i, j)$, $j = 1, \dots, S$, or, in matrix form: $\alpha_0 = \pi^S \pi_{r_0}^R$ and

$$\alpha_t = (A^S(\gamma(r_{t-1})))^T (a_{r_{t-1}r_t}^R(\cdot) \circ \alpha_{t-1}), \quad (6)$$

where the notation $a_{r_{t-1}r_t}^R(\cdot)$ means a column vector of the (r_{t-1}, r_t) -elements of the matrix $A^R(k)$ as k runs from 1 to S .⁴ It follows that

$$\ell(\mu) = \sum_{i=1}^S P(S_T = i, r_0, \dots, r_T \mid \mu) = \sum_i \alpha_T(i) = \mathbf{1}^T \alpha_T$$

can be computed with on the order of TS^2 computations.

An analogous concept that will be required later is the *backward variable*

$$\beta_t(i) := P(r_{t+1}, \dots, r_T \mid S_t = i, r_t), \quad (7)$$

which can also be computed via iteration: $\beta_T(j) = 1$, $\beta_{t-1}(j) = \sum_i \beta_t(i) a_{ji}^R(\gamma(r_t)) a_{r_{t-1}r_t}(j)$, or in matrix form: $\beta_T = \mathbf{1}$ and

$$\beta_{t-1} = (A^S(\gamma(r_t)))^T \beta_t \circ a_{r_{t-1}r_t}^R(\cdot). \quad (8)$$

3.2. Auxiliary function $Q(\mu, \mu')$

Let Λ denote the set of all cl3MCs. Λ is then bounded and convex if we define convex combinations of cl3MCs $\mu = (\pi^R, \pi^S, A^R(\cdot), A^S(\cdot); \Gamma)$ and $\nu = (\rho^R, \rho^S, B^R(\cdot), B^S(\cdot); \Gamma)$ as

$$\alpha\mu + (1-\alpha)\nu = (\alpha\pi^R + (1-\alpha)\rho^R, \alpha\pi^S + (1-\alpha)\rho^S, \alpha A^R + (1-\alpha)B^R, \alpha A^S + (1-\alpha)B^S; \Gamma).$$

See Epperlein et al. (2017) for details. Following Baum, Petri, Soules, and Weiss (1970), we define the auxiliary function $Q(\mu, \mu')$ of $\mu, \mu' \in \Lambda$ by

$$Q(\mu, \mu') := \sum_{s_0 \dots s_T} P(s_0, \dots, s_T, r_0, \dots, r_T \mid \mu) \cdot \log P(s_0, \dots, s_T, r_0, \dots, r_T \mid \mu'), \quad (9)$$

where $s_0, \dots, s_T$ run through the S^T possible sequences of the latent state S_t . If parameters μ'_i are zero where $\mu_i > 0$, then we can have the case $P(s_0, \dots, r_T \mid \mu') = 0$ and $P(s_0, \dots, r_T \mid \mu) > 0$; in this case $Q(\mu, \mu') := -\infty$. If $P(s_0, \dots, r_T \mid \mu) = P(s_0, \dots, r_T \mid \mu') = 0$, we set $Q(\mu, \mu') = 0$ which amounts to setting $0 \log(0) = 0$.

The following lemma establishes a representation for Q in terms of the elements of $\mu' = (\pi^{R'}, \pi^{S'}, A^{R'}, A^{S'}; \Gamma)$:

Lemma 2. *The function $Q(\mu, \mu')$ can be rewritten as*

$$Q(\mu, \mu') = \log \pi_{r_0}^{R'} \ell(\mu) + \sum_{i=1}^S \log \pi_i^{S'} \sum_{j=1}^S \xi_1(i, j) + \sum_{i,j=1}^S \sum_{t=1}^T L_{ij}(r_{t-1}, r_t) \xi_t(i, j) \quad (10)$$

⁴ Very much analogous to Matlab's colon notation, or slicing in numpy.

where $L_{ij}(m, n) := \log a_{ij}^{S'}(\gamma(m)) + \log a_{mn}^{R'}(i)$ and

$$\xi_t(i, j) := P(S_{t-1} = i, S_t = j, r_0, \dots, r_T \mid \mu) \quad (11)$$

can be computed as follows:

$$\xi_t(i, j) = \alpha_{t-1}(i) a_{r_{t-1}r_t}^R(i) a_{ij}^{S'}(\gamma(r_{t-1})) \beta_t(j), \quad (12)$$

and the variables carrying a $\bullet'$ constitute μ' .

The proof is relegated to Appendix B. Additionally, an application of Jensen's inequality yields (see Epperlein et al., 2017):

Lemma 3. *The improvement in log-likelihood satisfies the lower bound*

$$\ell(\mu)(\log \ell(\mu') - \log \ell(\mu)) \geq Q(\mu, \mu') - Q(\mu, \mu). \quad (13)$$

3.3. EM-algorithm for parameter estimation

The algorithm proceeds by maximising the lower bound on the log-likelihood improvement set forth in (13) at every iteration.

It should be clear from (10) that the best estimate of π^R is the r_0 -th canonical Euclidian basis vector e_{r_0} . The remaining parameters of μ can be iteratively estimated by repeatedly applying the following theorem:

Theorem 1. *The unique maximiser $\mu' = M(\mu)$ of $Q(\mu, \cdot)$ is given by*

$$\pi_i^{S'} = \frac{\sum_{j=1}^S \xi_1(i, j)}{\mathbf{1}^T \alpha_T} \quad (14a)$$

$$a_{ij}^{S'}(l) = \frac{\sum_{t: \gamma(r_{t-1})=l} \xi_t(i, j)}{\sum_{k=1}^S \sum_{t: \gamma(r_{t-1})=l} \xi_t(i, k)} \quad (14b)$$

$$a_{mn}^{R'}(i) = \frac{\sum_{k=1}^S \sum_{t: r_{t-1}=m, r_t=n} \xi_t(i, k)}{\sum_{v=1}^R \sum_{k=1}^S \sum_{t: r_{t-1}=m, r_t=v} \xi_t(i, k)}, \quad (14c)$$

where $i, j = 1, \dots, S$, $l = 1, \dots, p$, and $m, n = 1, \dots, R$.

The proof is given in Appendix B. Formulae (14) provide the basis for the EM-type parameter estimation algorithm for cl3MC μ : in its k th iteration, the E-step consists of computing $\xi_t(\cdot, \cdot)$ from the current estimate μ^k , and the M-step yields an updated estimate $\mu^{k+1} = M(\mu^k)$ with improved likelihood, see also the pseudocode in Algorithm 2 in the Appendix. Note that M has a unique fixed point, $\mu^\infty = \lim_k \mu^k$ which is, at the same time, a stationary point (possibly a local maxima) of the likelihood, see Epperlein et al. (2017) for details.

4. Examples

Here we illustrate the algorithm's efficacy in two scenarios: first with synthetic data, i.e. data generated from a cl3MC, denoted μ^{true} ; and second, in a toy example of a practical application, estimation of driver behaviour. In both cases, we assume that one decision-maker, specifically the matrix $A^R(2)$, and Γ are known. For implementation details, in particular how to avoid arithmetic underflow by scaling, and a pseudocode, see Appendix A and Algorithm 2 therein, and Epperlein et al. (2017); for details on the experimental procedures, see Appendix C.

4.1. Synthetic data

To explore the relationship between estimation error and number of samples, we repeated the following for several values $T \in [500, 75000]$: $N_e = 100$ cl3MCs with $R = 20$, $S = 2$ and $\Gamma = \{[R]\}$ (i.e. the open-loop case) were generated, and then a trajectory

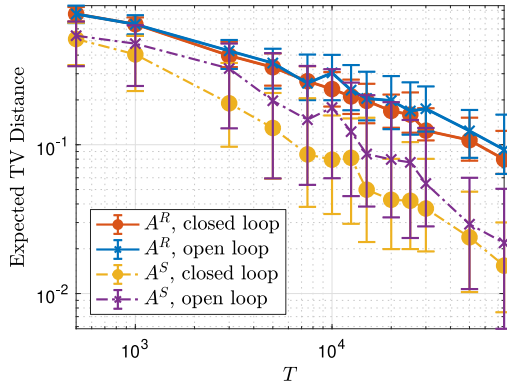


Fig. 2. Length of training sequences T vs. $\|A^{\bullet, \text{est}}(1) - A^{\bullet, \text{true}}(1)\|_{\text{exp}}$, defined in (16). Shown are the medians and whiskers for the quartiles.

of length T for each of them. We then ran the algorithm with random initial guesses π^S , A^S , and $A^R(1)$. The same was repeated for the same N_e cl3MCs, only that now, Γ was a randomly selected partition of order 2, so that $p = 2$, and a second random page $A^S(2)$ was added to A^S . In both cases, we assume $A^R(2)$ and Γ to be known. The modification to the algorithm is trivial: $A^R(2)$ is simply not re-estimated.

As illustrated in Figs. 2 and 3, we recover A^S and $A^R(1)$ to high accuracy for large enough T . “Accuracy” is hereby measured through statistical distances: since the transition matrices of Markov chains consist of probability distributions – row i being the distribution of the state following i – absolute or relative matrix norms are not a good measure of distance between Markov chains. Instead, we consider a statistical distance between the estimated and true probability distributions. One of the simplest such distances is the total variation (TV) distance (see e.g. Levin, Peres, & Wilmer, 2009, Ch. 4), which is given by the maximal difference in probability for any event between two distributions. For probability distributions f and g over a discrete space Ω , this is simply

$$\|f - g\|_{\text{TV}} = \max_{A \subseteq \Omega} f(A) - g(A) = \frac{1}{2} \sum_{\omega \in \Omega} |f(\omega) - g(\omega)|.$$

We consider here two applications of TV distance to Markov chains. The first is to take the TV distance between the stationary distributions, which concretely amounts to considering the subset ρ of the state space $[R]$ such that $P(X_t \in \rho \mid A^{R, \text{est}}(1)) - P(X_t \in \rho \mid A^{R, \text{true}}(1))$ is maximised (for large enough times t such that the stationary distribution is reached). If we let ψ^{true} and ψ^{est} denote the stationary distributions, then

$$\|A^{R, \text{est}}(1) - A^{R, \text{true}}(1)\|_{\text{stat}} := \|\psi^{\text{est}} - \psi^{\text{true}}\|_{\text{TV}}. \quad (15)$$

However, this is a coarse measure: different Markov chains can have equal stationary distributions. Hence, the second metric incorporates the distance between the individual rows by considering the expectation (under the true stationary distribution ψ^{true}) of the TV distance between the estimated and the true row; this equals the sum of the distances between the true and estimated transition probabilities from all states i , weighted by the probability of being in state i :

$$\|A^{R, \text{est}}(1) - A^{R, \text{true}}(1)\|_{\text{exp}} := \sum_i \psi_i^{\text{true}} \|A_{i:}^{R, \text{est}}(1) - A_{i:}^{R, \text{true}}(1)\|_{\text{TV}}, \quad (16)$$

where $M_{i:}$ denotes the i th row of matrix M .

The effect of T on the accuracy is explored in Fig. 2. The error appears to decay as a power of T , however this is simply an

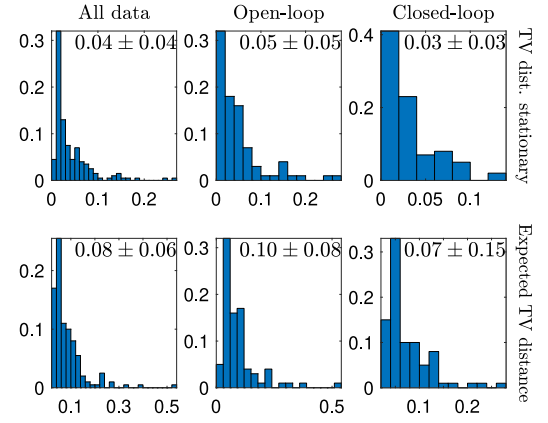


Fig. 3. The upper row shows the distribution of $\|A^{R, \text{est}}(1) - A^{R, \text{true}}(1)\|_{\text{stat}}$, see (15), first for all $2N_e = 200$ pairs of μ^{true} and μ^{est} , and then for the N_e open-loop and N_e closed-loop cases. In the bottom row, the same is shown for the metric $\|A^{R, \text{est}}(1) - A^{R, \text{true}}(1)\|_{\text{exp}}$, defined in (16).

observation; a theoretical analysis of the sample complexity and decay rates is part of future work to be done.

For a representative value of $T = 5 \cdot 10^4$, Fig. 3 drills down further into the experimental results; the distance for both introduced metrics is often below 10%, but we also observe severe outliers. Note that we show $A^R(1)$ only, the analysis and results for $A^S(\cdot)$ are analogous and are hence omitted.

4.2. A model of driver behaviour

Recent research, e.g. Epperlein et al. (2018), Krumm (2008), Lassoued et al. (2017) and Simmons, Browning, Zhang, and Sadekar (2006), suggests that Markov-based models are good approximations of driver behaviour and can be used e.g. for route prediction. Here, we illustrate how cl3MCs can be used to identify a driver's preferences when some trips are planned by a recommender system, whose preferences are known, while the other trips are planned by the driver.

Specifically, consider the map in the left panel of Fig. 4, which depicts a (very small toy) model of a driver's possible routes from origin “O” to destination “D”. The houses, as an example, correspond to schools, that should be avoided in the hour before classes start and after classes end for the day, so there is a route past them and one around them. We assume that if a trip falls into that time frame, the recommender takes over and, with known probabilities, routes the driver either past or around each school; these probabilities make up $A^R(2)$. Otherwise, the driver follows his/her preferences, which constitute $A^R(1)$; this is the matrix we would like to estimate. We generated $N_e = 50$ sets of data by simulating $N_t = 200$ trips on the graph shown in the right panel of Fig. 4; this is the line graph of the map, where each road segment corresponds to a node, and an edge goes from node i to node j iff it is possible to turn into road segment j from i . Each trip has a probability of $p_r = 0.3$ to be planned by the recommender. If a trip was planned by the recommender (resp. driver), a trajectory was generated by a Markov chain with transition matrix $A^R(2)$ (resp. $A^R(1)$) originating in node 1 and terminating when returning to node 1.

For estimation in the cl3MC framework, all trips are then concatenated to form one long trajectory and A^S and $A^R(1)$ are estimated for an ol3MC, i.e. for $\Gamma = \{[9]\}$. $A^{R, \text{est}}(1)$ is then an estimate of the driver preferences. The results are shown in the first column of Fig. 5 and are satisfactory already; however, we can leverage the closed-loop framework to include the additional knowledge that the decision maker (i.e. the page of A^R used) can only change

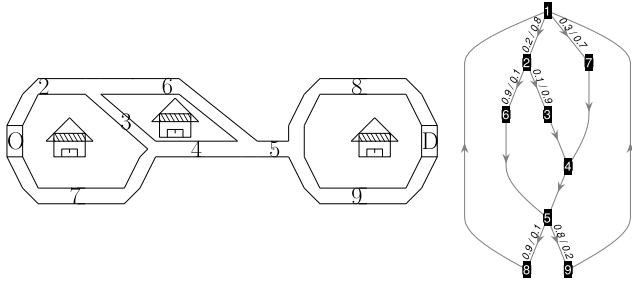


Fig. 4. The map of our small toy model and its abstraction as the line graph of the road model. The direction of traffic is from left to right only. The origin and destination are merged into node 1. The weights denote the transition probabilities for the driver and the recommender system. When there is no weight, then the transition probability is 1 for both.

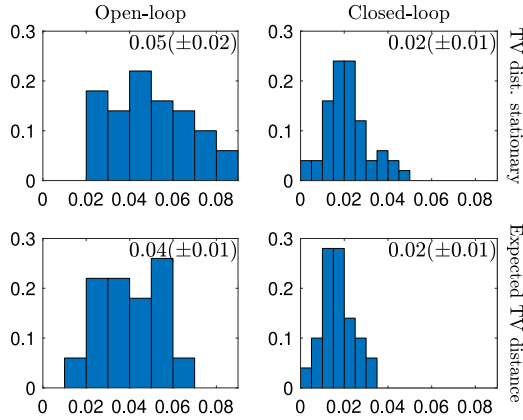


Fig. 5. Results from estimating the driver preferences in Section 4.2. Assuming open loop yields acceptable results; adding the information that only full trips are planned by either driver or recommender improves accuracy considerably. Mean and standard deviation are given.

after a trip is finished. Because the decision which page of A^R to use at time t is made at $t - 1$, see (1), this means we have to allow for the state of $\mathbf{S}$ to change on the road segments *prior* to reaching the destination. We hence let $\Gamma = \{\{8, 9\}, \{1, \dots, 7\}\}$ and $A^S(2) = I_2$. $A^S(1)$ needs to be identified. The results are shown in Fig. 5. Additionally, we can interpret $\psi_2^{S, \text{est}}$, the second element of the stationary distribution of $A^{S, \text{est}}(1)$ as an estimate of p_r . For the open-loop case, we obtain $0.352(\pm 0.0583)$, whereas the cl3MC estimation yields $0.295(\pm 0.044)$.

5. Concluding remarks

We consider the identification of user models acting under the influence of one or more recommender systems. As we have already discussed, actuating behavioural change affects the original model underpinning the predictor, leading to a biased user models. Given this background, the specific contribution of this paper is to develop techniques in which unbiased estimates of user behaviour can be recovered in the case where recommenders, users, and switching between them can be parameterised in a Markovian manner, and where users and recommenders form part of a feedback system. Examples are given to present the efficacy of our approach.

Acknowledgements

The authors would like to thank Ming-Ming Liu and Yingqi Gu (University College Dublin) for help with the numerical

experiments, and Giovanni Russo and Jakub Mareček (IBM Research Ireland) for valuable discussions.

This work has been conducted within the ENABLE-S3 project that has received funding from the ECSEL joint undertaking under grant agreement NO 692455. This joint undertaking receives support from the European Union's HORIZON 2020 Research and Innovation programme and Austria, Denmark, Germany, Finland, Czech Republic, Italy, Spain, Portugal, Poland, Ireland, Belgium, France, Netherlands, United Kingdom, Slovakia, Norway.

Robert Shorten was also partially supported by SFI grant 16/IA/4610.

Appendix A. Scaling issues

Since the computations of α_t and β_{T-t} according to (6) and (8) involve multiplications of on the order of $2t$ numbers less than 1, for large T , they will be close to, or below, machine precision. The re-estimation (14) then requires division of very small numbers, which of course should be avoided. To mitigate these issues, scale α_t to sum up to 1:

$$c_t := \mathbf{1}^\top \alpha_t \quad \hat{\alpha}_t := \alpha_t / c_t.$$

The update can be done in two compact steps:

$$\begin{aligned} \hat{\alpha}'_t &= (A^S(\gamma(r_{t-1})))^\top (a_{r_{t-1}r_t}^R(\cdot) \circ \hat{\alpha}_{t-1}) \\ \hat{\alpha}_t &= \frac{\hat{\alpha}'_t}{\mathbf{1}^\top \hat{\alpha}'_t}. \end{aligned} \quad (\text{A.1})$$

Also note that $\log c_t = \log c_{t-1} + \log \mathbf{1}^\top \hat{\alpha}'_t$ and $\log \ell(\mu) = \log(\mathbf{1}^\top \alpha_T) = \log(c_T)$; the likelihood $\ell(\mu)$ is not computed anymore, only the log-likelihood. Since the backwards variables β_{T-t} can be expected to be of similar order as α_t they are scaled using the same scaling factors c_t :

$$\hat{\beta}_{T-t} := \beta_{T-t} / c_t. \quad (\text{A.2})$$

We then compute the scaled versions of $\xi_t(i, j)$ in similar fashion

$$\begin{aligned} \hat{\xi}'_t(i, j) &= \hat{\alpha}_{t-1}(i) a_{r_{t-1}r_t}^R(i) a_{ij}^S(\gamma(r_{t-1})) \hat{\beta}_t(j) \\ \hat{\xi}_t &= \frac{\hat{\xi}'_t}{\mathbf{1}^\top \hat{\xi}'_t \mathbf{1}} = \frac{\xi_t}{\mathbf{1}^\top \alpha_T} \end{aligned} \quad (\text{A.3})$$

and note that upon substituting $\xi_t = (\mathbf{1}^\top \alpha_T) \hat{\xi}_t$ in (14), $\mathbf{1}^\top \alpha_T$ cancels everywhere, and we arrive at the rescaled re-estimation equations

$$\pi_i^{S'} = \sum_{j=1}^S \hat{\xi}_1(i, j) \quad (\text{A.4a})$$

$$a_{ij}^{S'}(l) = \frac{\sum_{t: \gamma(r_{t-1})=l} \hat{\xi}_t(i, j)}{\sum_{k=1}^S \sum_{t: \gamma(r_{t-1})=l} \hat{\xi}_t(i, k)} \quad (\text{A.4b})$$

$$a_{mn}^{R'}(i) = \frac{\sum_{k=1}^S \sum_{t: r_{t-1}=m, r_t=n} \hat{\xi}_t(i, k)}{\sum_{k=1}^S \sum_{t: r_{t-1}=m} \hat{\xi}_t(i, k)}. \quad (\text{A.4c})$$

For a more detailed derivation, see again Epperlein et al. (2017).

Appendix B. Proofs

Proof of Lemma 2. Formula (12) follows directly from the definitions of α_{t-1} , a_{mn}^R , a_{ij}^S and β_t . Now let summation indices s_t always run from 1 to S . First, we rewrite the second line of (9): given the

Algorithm 2 EM parameter estimation for cl3MCs.

```

1: procedure EM-ESTIMATION(  $\mathcal{O} = (r_0 \dots r_T), \mu = (\pi^R, \pi^S, A^R, A^S; \Gamma); \epsilon > 0, N_{\max} > 0$  )
   Initialization
2:    $N_{\text{iter}} := 0$                                  $\delta := \infty$ 
3:    $\pi^R \leftarrow e_{r_0}$                                  $\triangleright$  See remark before Theorem 1
   Iteration
4:   while  $N_{\text{iter}} \leq N_{\max}$  and  $\delta > \epsilon$  do
     Forward-backward variables from  $\mu$ 
5:      $\alpha \leftarrow (6)$      $\beta \leftarrow (8)$      $\xi \leftarrow (12)$ 
       or  $\hat{\alpha} \leftarrow (A.1)$   $\hat{\beta} \leftarrow (A.2)$   $\hat{\xi} \leftarrow (A.3)$ 
     Reestimation
6:      $(\pi^{S'}, A^{R'}, A^{S'}) \leftarrow (14)$  or  $(A.4)$ 
7:      $\mu' \leftarrow (\pi^R, \pi^{S'}, A^{R'}, A^{S'}; \Gamma)$ 
     Check relative changes
8:     for  $v \in \{\pi^S, A^R, A^S\}$  do
9:        $\delta_v \leftarrow (\|v - v'\|_F) / \|v\|_F$ 
10:     $\delta_\ell \leftarrow |\ell(\mu') - \ell(\mu)| / |\ell(\mu)|$ 
11:     $\delta \leftarrow \max\{\delta_v\}$ 
   Update
12:    $N_{\text{iter}} \leftarrow N_{\text{iter}} + 1$ 
13:    $(\pi^S, A^R, A^S) \leftarrow (\pi^{S'}, A^{R'}, A^{S'})$      $\mu \leftarrow \mu'$ 
14: return  $\mu$ 

```

definition of $L_{ij}(k, \ell)$ and (3) we get

$$\log P(s_0, \dots, s_T, r_0, \dots, r_T \mid \mu') = \log \pi_{r_0}^{R'} + \log \pi_{s_0}^{S'} + \sum_{t=1}^T L_{s_{t-1}s_t}(r_{t-1}, r_t).$$

We substitute this back into (9) to get $Q(\mu, \mu') = Q_1 + Q_2 + Q_3$, with $Q_1 = \sum_{s_0 \dots s_T} P(s_0, \dots, s_T, r_0, \dots, r_T \mid \mu) \cdot \log \pi_{r_0}^{R'} = \ell(\mu) \log \pi_{r_0}^{R'}$; for Q_2 and Q_3 , note the “marginalisation”

$$\xi_t(i, j) = \sum_{s_0 \dots s_{t-2}, s_{t+1} \dots s_T} P(s_0, \dots, s_{t-2}, s_{t-1} = i, s_t = j, \dots, s_T, r_0, \dots, r_T \mid \mu),$$

where the sum runs over the states of the latent chain $\mathbf{S}$ before time instant $t - 1$ and after time instant t . Then

$$Q_2 = \sum_{s_0 \dots s_T} P(s_0, \dots, s_T, r_0, \dots, r_T \mid \mu) \log \pi_{s_0}^{S'} = \sum_{i,j=1}^S \log \pi_i^{S'} \sum_{s_2 \dots s_T} P(s_0 = i, s_1 = j, s_2, \dots, s_T \mid \mu) = \sum_{i=1}^S \log \pi_i^{S'} \sum_{j=1}^S \xi_1(i, j)$$

and

$$Q_3 = \sum_{s_0 \dots s_T} P(s_0, \dots, r_T \mid \mu) \sum_{t=1}^T L_{s_{t-1}s_t}(r_{t-1}, r_t) = \sum_{t=1}^T \sum_{s_{t-1}, s_t} L_{s_{t-1}s_t}(r_{t-1}, r_t) \cdot P(s_{t-1} = s_{t-1}, s_t = s_t, r_0, \dots, r_T \mid \mu) = \sum_{t=1}^T \sum_{i=1}^S \sum_{j=1}^S L_{ij}(r_{t-1}, r_t) \xi_t(i, j).$$

This completes the proof. ■

Proof of Theorem 1. From the remark before the theorem, it should be clear that we can ignore the first term in (10) in the maximisation. Consider $\mu \in \Lambda$ as fixed, and define $\tilde{\mu} \mapsto W(\tilde{\mu}) := Q(\mu, \tilde{\mu})$. We claim that W has the unique global maximum point $\mu' \in \Lambda$. Note that if $\mu \in \Lambda$ then it may have zero components, say $\mu_{i_1} = \dots = \mu_{i_d} = 0$. Then the logarithms of the corresponding components $\tilde{\mu}_{i_1} \dots \tilde{\mu}_{i_d}$ of $\tilde{\mu}$ in W are multiplied by 0 so that these components do not change W . However, if we fix all the components of $\tilde{\mu}$ but $\tilde{\mu}_{i_1}$ and any other component $\tilde{\mu}_k$ such that $k \notin \{i_1 \dots i_d\}$, and k is such that $\tilde{\mu}_{i_1}$ and $\tilde{\mu}_k$ are in π^R , or are in the same row of A^R or A^S , then increasing $\tilde{\mu}_{i_1}$ will decrease $\tilde{\mu}_k$ (to meet the stochasticity constraints). As a result, the $\log \tilde{\mu}_k$ will decrease causing W to decrease. Hence, the maximum of W is attained in the set $\tilde{\Lambda} := \{\tilde{\mu} \in \Lambda : \tilde{\mu}_{i_1} = \dots = \tilde{\mu}_{i_d} = 0\}$. Let $\tilde{W}$ be the restriction of W to $\tilde{\Lambda}$. Now $\tilde{W}$ is a conical sum of logarithms of all $R + S + SR^2 + pS^2 - d$ independent components of $\tilde{\mu}$, hence strictly concave function on a convex compact set $\tilde{\Lambda}$. Hence, $\tilde{W}$ has a unique maximum point in $\tilde{\Lambda}$ which coincides with the unique global maximum point μ' of W in Λ .

Let us prove that $\mu' = M(\mu)$. As noted above, $\mu_{i_1} = \dots = \mu_{i_d} = 0$ implies that $\mu'_{i_1} = \dots = \mu'_{i_d} = 0$, and we stress that the same property holds true for $M(\mu)$: If π_i^S is 0 it follows from (6) that $\alpha_0(i) = 0$ and from (12) we get $\xi_1(i, j) = 0$. By (14a), $\pi_i^{S'} = 0$ as well. If we have $a_{ij}^{S'}(l) = 0$, then again from (12), we see that $\xi_t(i, j) = 0$ for all t with $r_{t-1} = l$, and (14b) yields $a_{ij}^{S'}(l) = 0$. Similarly, $a_{mn}^R(i) = 0$ leads to $\xi_t(i, j) = 0$ whenever $r_{t-1} = m$ and $r_t = n$, independently of j , so $a_{mn}^{R'}(i) = 0$, too. Hence $M(\tilde{\Lambda}) = \tilde{\Lambda}$, and $\pi_i^{S'}$, $a_{ij}^{S'}(l)$ and $a_{mn}^{R'}(i)$ defined by (14) are positive if the corresponding components of μ are. On the other hand, $\mu'_k \geq 0$ if $\mu_k > 0$ as otherwise $W(\mu') = -\infty$, and so the gradient of $\tilde{W}$ is well-defined at μ' . In fact, for positive $\tilde{\pi}_i^S$, $\tilde{a}_{ij}^{S'}(l)$ and $\tilde{a}_{mn}^R(i)$:

$$\begin{aligned} \frac{\partial \tilde{W}}{\partial \tilde{\pi}_i^S} &= \frac{\sum_{j=1}^S \xi_1(i, j)}{\tilde{\pi}_i^S} \\ \frac{\partial \tilde{W}}{\partial \tilde{a}_{ij}^{S'}(l)} &= \frac{\sum_{t: \gamma(r_{t-1})=l} \xi_t(i, j)}{\tilde{a}_{ij}^{S'}(l)} \\ \frac{\partial \tilde{W}}{\partial \tilde{a}_{mn}^R(i)} &= \frac{\sum_{k=1}^S \sum_{t: r_{t-1}=m, r_t=n} \xi_t(i, k)}{\tilde{a}_{mn}^R(i)} \end{aligned}$$

By e.g. Bertsekas (1999, p.113, Prop. 2.1.2), it is necessary and sufficient for μ' to satisfy the inequality

$$(\nabla \tilde{W}(\tilde{\mu})|_{\tilde{\mu}=\mu'})^T \tilde{\mu} \leq (\nabla \tilde{W}(\tilde{\mu})|_{\tilde{\mu}=\mu'})^T \mu' \quad \forall \tilde{\mu} \in \tilde{\Lambda}. \quad (\text{B.1})$$

We stress that the r.h.s. of (B.1) is independent of μ' as

$$(\nabla \tilde{W}(\tilde{\mu})|_{\tilde{\mu}=\mu'})^T \mu' = (2T + 1)\ell(\mu).$$

Since $\pi_i^{S'}$, $a_{ij}^{S'}(l)$ and $a_{mn}^{R'}(i)$ defined by (14) are positive if the corresponding components of μ are so, the gradient of $\tilde{W}$ is well-defined at $M(\mu)$. Take any $\tilde{\mu} \in \tilde{\Lambda}$ and compute:

$$\begin{aligned} \tilde{\pi}_i^S \frac{\partial \tilde{W}}{\partial \tilde{\pi}_i^S}(\mu') &= \tilde{\pi}_i^S \sum_{i=1}^S \sum_{j=1}^S \xi_1(i, j) \\ \tilde{a}_{ij}^{S'}(l) \frac{\partial \tilde{W}}{\partial \tilde{a}_{ij}^{S'}(l)}(\mu') &= \tilde{a}_{ij}^{S'}(l) \sum_{k=1}^S \sum_{t: \gamma(r_{t-1})=l} \xi_t(i, k) \\ \tilde{a}_{mn}^R(i) \frac{\partial \tilde{W}}{\partial \tilde{a}_{mn}^R(i)}(\mu') &= \tilde{a}_{mn}^R(i) \sum_{n=1}^R \sum_{k=1}^S \sum_{t: r_{t-1}=m, r_t=n} \xi_t(i, k) \end{aligned}$$

so that, by stochasticity constraint, we get (Epperlein et al., 2017):

$$(\nabla \tilde{W}(\tilde{\mu})|_{\tilde{\mu}=\mu'})^T \tilde{\mu} = (2T + 1)\ell(\mu)$$

Hence, $M(\mu)$ defined by (14) satisfies (B.1) with equality for any $\tilde{\mu} \in \tilde{\Lambda}$. This completes the proof. ■

Appendix C. Experimental details

The detailed steps in generating the data in Section 4.1 are as follows. We consider $T \in \{500, 1000, 3000, 5000, 7500, 10000, 12500, 15000, 20000, 25000, 30000, 50000, 75000\}$ and begin at $i = 0$. Then, for each of the $N_T = 13$ values of T , the following is repeated N_e times: (1) generate a pair of $R \times R$ row-stochastic matrices $(A^R(1), A^R(2))$ by selecting one “dominant element” per row and setting it to a random number uniformly distributed in $[0.5, 1]$ (this is done to ensure that the two matrices in a pair are sufficiently different) and filling the remaining elements with uniformly random numbers; (2) generate a $S \times S$ row-stochastic matrix $A^S(1)$ with uniformly random entries; (3) generate initial probability vectors $\pi^S \in [0, 1]^S$ and $\pi^R \in [0, 1]^R$ with uniformly random entries; (4) generate a trajectory of length T from the cl3MC $\mu_o = (\pi^S, \pi^R, A^S(1), A^R; [R])$ (this is the open-loop case); (5) estimate parameters of μ_o by running Algorithm 2 (initialised with uniformly random guesses for unknown parameters) until convergence and record the distance measures defined in Section 4.1; (6) generate an additional $S \times S$ row-stochastic matrix $A^S(2)$ with uniformly random entries; (7) generate a random partition Γ of $[R]$ by first randomly permuting $[R]$ and then splitting it after a random index between 1 and $R-2$; (8) generate a trajectory of length T from the cl3MC $\mu_c = (\pi^S, \pi^R, A^S, A^R; \Gamma)$ (this is the open-loop case); (9) estimate parameters of μ_c by running Algorithm 2 (initialised with uniformly random guesses for unknown parameters) until convergence and record the distance measures defined in Section 4.1. Note that we assume $A^R(2)$ to be known and it is not estimated. Concretely that means that the initial guess of $A^R(2)$ is set to the true value of $A^R(2)$ and $A^R(2)$ is excluded from the reestimation steps in Algorithm 2. The parameters used in Algorithm 2 are $\varepsilon = 10^{-5}$, a typical, empirical choice for relative tolerances, and $N_{max} = 2000$, at which point the algorithm has typically long converged (in our experiments, N_{max} was never reached). Once the experiment terminates, we have $2N_e N_T = 2600$ (resp. $3N_e N_T = 3900$) different values for each of the distance metrics for A^R (resp. A^S), which are visualised in Figs. 2 and 3.

For Section 4.2, the matrices $(A^R(1), A^R(2))$ are fixed to the ones indicated in Fig. 4. One run then consisted of repeating N_t times the following: (1) draw a number from the uniform distribution on $[0, 1]$ and if it is less than p , set $s = 1$, else $s = 2$; (2) using the transition probabilities in $A^R(s)$, generate a trajectory from initial state O until state D is reached. We then concatenated all of those trajectories, identifying states O and D as state 1, e.g. if there were only two trajectories, $(O, 2, 6, 5, 9, D)$ and $(O, 7, 4, 5, 8, D)$ they are concatenated to $(1, 2, 6, 5, 9, 1, 7, 4, 5, 8, 1)$. Algorithm 2 is then run on that single trajectory with the same parameters as above, again assuming $A^R(2)$ as known, two times: once assuming open-loop, i.e. $\Gamma = [9]$, and once assuming closed-loop with $\Gamma = \{\{8, 9\}, \{1, \dots, 7\}\}$ and $A^S(2) = I_2$. Then the distance metrics between estimates and true values are computed. This concluded one run; we collected N_e runs, yielding $2N_e = 100$ numbers for each $A^R(1)$ and $A^S(1)$. The numbers for $A^R(1)$ are shown in Fig. 5.

References

- Angrist, J., & Pischke, J. (2008). *Mostly harmless econometrics: An empiricist's companion*. Princeton University Press.
- Baum, Y., Petri, T., Soules, G., & Weiss, N. (1970). A maximization technique occurring in the statistical analysis of probabilistic functions of Markov chains. *The Annals of Mathematical Statistics*, 41, 164–171.
- Bertsekas, D. P. (1999). *Nonlinear programming*. Athena Scientific Belmont.
- Blackwell, D., & Koopmans, L. (1957). On the identifiability problem for functions of finite Markov chains. *The Annals of Mathematical Statistics*, 1011–1015.
- Bottou, L., Peters, J., Quiñero Candela, J., Charles, D. X., Chickering, D. M., Portugaly, E., et al. (2013). Counterfactual reasoning and learning systems: The example of computational advertising. *Journal of Machine Learning Research*, 14, 3207–3260. URL <http://dl.acm.org/citation.cfm?id=25677092567766>.
- Cosley, D., Lam, S. K., Albert, I., Konstan, J. A., & Riedl, J. (2003). Is seeing believing? How recommender system interfaces affect users' opinions. In *Proceedings of the SIGCHI conference on human factors in computing systems* (pp. 585–592). ACM.
- Costa, O. L. V., Fragoso, M. D., & Marques, R. P. (2006). *Discrete-time Markov jump linear systems*. Springer Science & Business Media.
- Crisostomi, E., Shorten, R., & Wirth, F. (2016). Smart cities: A golden age for control theory? [industry perspective]. *IEEE Technology and Society Magazine*, 35, 23–24.
- Ephraim, Y., & Roberts, W. J. J. (2009). An EM algorithm for Markov modulated Markov processes. *IEEE Transactions on Signal Processing*, 57, 463–470. <http://dx.doi.org/10.1109/TSP.2008.2007919>.
- Epperlein, J. P., Monteil, J., Liu, M., Gu, Y., Zhuk, S., & Shorten, R. (2018). Bayesian classifier for route prediction with Markov chains. *IEEE International Conference on Intelligent Transportation Systems*, Preprint available arXiv:1808.10705.
- Epperlein, J., Shorten, R., & Zhuk, S. (2017). Learning Markov models from closed loop data-sets. ArXiv e-prints arXiv:1706.06359v2.
- Forssell, U., & Ljung, L. (1999). Closed-loop identification revisited. *Automatica*, 35, 1215–1241. [http://dx.doi.org/10.1016/S0005-1098\(99\)00022-9](http://dx.doi.org/10.1016/S0005-1098(99)00022-9), URL <http://www.sciencedirect.com/science/article/pii/S0005109899000229>.
- Gilbert, E. (1959). On the identifiability problem for functions of finite Markov chains. *The Annals of Mathematical Statistics*, 30, 688–697.
- Jacobs, R. A., Jordan, M. I., Nowlan, S. J., & Hinton, G. E. (1991). Adaptive mixtures of local experts. *Neural Computation*, 3, 79–87.
- Krumm, J. (2008). A Markov model for driver turn prediction. Technical Report. SAE Technical Paper.
- Lange, T., & Rahbek, A. (2009). An introduction to regime switching time series models. In *Handbook of financial time series*. Springer.
- Lassoued, Y., Monteil, J., Gu, Y., Russo, G., Shorten, R., & Mevissen, M. (2017). Hidden Markov model for route and destination prediction. In *IEEE international conference on intelligent transportation systems*.
- Lazer, D., Kennedy, R., King, G., & Vespignani, A. (2014). The parable of Google Flu: Traps in big data analysis. *Science*, 343, 1203–1205.
- Levin, D. A., Peres, Y., & Wilmer, E. L. (2009). *Markov chains and mixing times* (2nd ed.). American Mathematical Soc..
- Loram, I. D., Gollee, H., Lakie, M., & Gawthrop, P. J. (2011). Human control of an inverted pendulum: Is continuous control necessary? Is intermittent control effective? Is intermittent control physiological? *The Journal of Physiology*, 589, 307–324. <http://dx.doi.org/10.1113/jphysiol.2010.194712>.
- Meila, M., & Jordan, M. I. (1996). Markov mixtures of experts. In R. Murray-Smith, & T. A. Johansen (Eds.), *Multiple model approaches to nonlinear modelling and control*. Taylor and Francis.
- Norton, J. (2009). *Dover books on electrical engineering series, An introduction to identification*. Dover Publications, URL https://books.google.ie/books?id=eyHC7751n_cC.
- Pearl, J. (2000). *Causality: Models, reasoning, and inference*. New York: Cambridge University Press.
- Petrie, T. (1969). Probabilistic functions of finite state Markov chains. *The Annals of Mathematical Statistics*, 40, 97–115.
- Rabiner, L. R. (1989). A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77, 257–286. <http://dx.doi.org/10.1109/5.18626>.
- Schlote, A., Chen, B., & Shorten, R. (2015). On closed loop bicycle availability prediction. *IEEE Transactions on Intelligent Transportation Systems*, 16, 1449–1555.
- Schlote, A., King, C., Crisostomi, E., & Shorten, R. (2014). Delay-tolerant stochastic algorithms for parking space assignment. *IEEE Transactions on Intelligent Transportation Systems*, 15, 1922–1935.
- Simmons, R., Browning, B., Zhang, Y., & Sadekar, V. (2006). Learning to predict driver route and destination intent. In *2006 IEEE intelligent transportation systems conference* (pp. 127–132). <http://dx.doi.org/10.1109/ITSC.2006.1706730>.
- Sinha, A., Gleich, D., & Ramani, K. (2016). Deconvolving feedback loops in recommender systems. In *Proceedings of NIPS*. Barcelona, Spain: ArXiv:1703.01049.
- Söderström, T., & Stoica, P. (1989). *Prentice hall international series in systems and control engineering, System identification*. Prentice Hall, URL https://books.google.ie/books?id=X_xQAAAMAAJ.
- Stock, James H., & Trebbi, F. (2003). Retrospectives: Who invented instrumental variable regression? *Journal of Economic Perspectives*, 17, 177–194.
- Van Den Hof, P. M., & Schrama, R. J. (1995). Identification and control – closed-loop issues. *Automatica*, 31, 1751–1770.
- Varian, H. R. (2016). Causal inference in economics and marketing. *Proceedings of the National Academy of Sciences of the United States of America*, 7310–7315.



diagnostics and distributed parameter systems.

Jonathan P. Epperlein was awarded the Dipl.-Ing. degree in Engineering Cybernetics from Otto-von-Guericke Universität in Magdeburg, Germany in 2008, and in 2014 (resp. 20011) the Ph.D. (resp. M.Sc.) in Electrical and Computer Engineering from the University of California, Santa Barbara. He has also spent time at the Hamilton Institute in Maynooth, Ireland, Yale University, and Harvard Medical School. Since 2016, he has been a Research Staff Member at IBM Research, Ireland, and his current research interests are centred around closed-loop issues in Machine Learning, bio-physical modelling in cancer



Sergiy Zhuk is a Research Staff Member and Master Inventor at IBM Research-Ireland and adjunct assistant professor at University College Dublin. Sergiy holds a Ph.D. in Applied Mathematics from Taras Shevchenko National University of Kyiv, Ukraine. In 2009 Sergiy was granted ERCIM/Marie Curie fellowship with Centrum Wiskunde Informatica (The Netherlands) and INRIA Paris-Rocquencourt research center (France) from 2009 to 2011. In 2010 he won an award for young researchers from the Ukrainian National Academy of Science. Sergiy has been a visiting scholar at Inria Paris research center

and Mathematical institute of Utrecht University. Sergiy joined IBM Research in 2012. His main research interests include nonlinear filtering for chaotic dynamical systems, inverse modelling in biophysics, detectability for convolutional neural networks and partially observed Markov decision processes.



Robert Shorten is Professor of Control Engineering and Decision Science at UCD. He was a cofounder of the Hamilton Institute at Maynooth University, worked at Daimler-Benz Research in Berlin, and led the Optimisation and Control team at IBM Research Smart Cities Research Lab in Dublin Ireland. He has been a visiting professor at TU Berlin and a research visitor at Yale University and Technion. He is the Irish member of the European Union Control Association assembly, a member of the IEEE Control Systems Society Technical Group on Smart Cities, and a member of the IFAC Technical Committees

for Automotive Control, and for Discrete Event and Hybrid Systems. He is a co-author of the recently published books AIMD Dynamics and Distributed Resource Allocation (SIAM 2016) and Electric and Plug-in Vehicle Networks: Optimisation and Control (CRC Press, Taylor and Francis Group, 2017). His research interests include computer networking, automotive research, collaborative mobility (including smart transportation and electric vehicles), as well as basic control theory and linear algebra. His main field of theoretical research has been the study of hybrid dynamical systems.